

EXAMEN-PRACTICAS.pdf



Sr_Aprobados



Programación y Diseño Orientado a Objetos



2º Grado en Ingeniería Informática



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada



La mejor escuela de negocios en energía, sostenibilidad y medio ambiente de España.

Más información
www.eoi.es

Formamos
talento para un futuro
Sostenible



100% Empleabilidad



Modalidad: Presencial u online



**Programa de Becas,
Bonificaciones y Descuentos**

CAMBIA TU VIDA ESTE VERANO

EXAMEN DE PRÁCTICAS PDOO – Prácticas 1,2 y 3. Grupo B1

16-11-2023

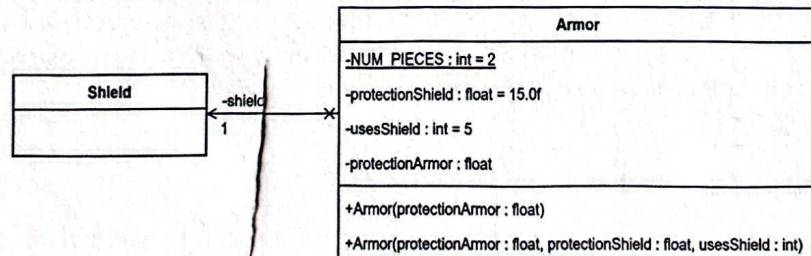
Nombre, Apellidos y DNI: _____

Se debe devolver esta hoja al profesor. Quien no lo haga será calificado con "No presentado"

1. (20%) Escribe V o F junto a cada afirmación según consideres que es verdadera o falsa. Es obligatorio justificar las respuestas falsas para que se consideren correctas. Cada respuesta correcta suma 0,4 puntos. Cada respuesta errónea resta 0,4 puntos hasta llegar como mínimo a cero puntos. Las respuestas en blanco ni suman ni restan.

1.	Todos los atributos y métodos de Dice son de clase, y se configuran en Java usando la directiva final.
2.	En el método configureLabyrinth se configura la posición en el laberinto de los monstruos, jugadores y obstáculos.
3.	El método dead en la clase Monster verifica si la salud del monstruo es menor o igual a cero.
4.	La función addBlock permite agregar, a la misma vez, más de un bloque 'X' con diferente orientación en el laberinto.
5.	La función putPlayer siempre devuelve un objeto Monster.

2. (30%) (En JAVA y Ruby) Modificar lo que se estime necesario en el proyecto Irrgarten para incorporar la nueva clase Armor y las relaciones del siguiente diagrama de clases. Se deben implementar todos los métodos propuestos en el diagrama. **Obligatorio:** Añadir al final del archivo de la clase Armor (entre comentarios) una explicación razonada de los cambios y decisiones realizadas.



3. (50%) (En Ruby) Crear un programa de prueba de la práctica realizada. Para ello,
- Modificar lo que sea necesario para que los jugadores NO se posicionen aleatoriamente, sino en una celda fija del laberinto. Crear un laberinto de tamaño 5x5 con la casilla de salida en la posición (1,1). Crear un monstruo, y un jugador (lo suficientemente débil para no poder vencer al monstruo). Completar la última fila del laberinto con obstáculos, con solo una llamada al método especificado para ello.
 - Añadir al proyecto una nueva clase pública, denominada ExamenTest y que contenga un método main.
 - Configurar el proyecto para que cuando se ejecute el proyecto se ejecute solamente este nuevo main.
 - Añadir al main las instrucciones necesarias para:
 - Instanciar un nuevo objeto Game con solo 1 jugador.
 - Posicionar al jugador en la casilla (0,0) y al monstruo en la casilla (1,0).
 - Solicitarle al juego un nuevo GameState y mostrar todo su contenido por pantalla.
 - Hacer que el jugador avance hacia el monstruo.
 - Solicitarle al juego un GameState y mostrar todo su contenido por pantalla.
 - Hacer que el jugador avance hacia la salida.
 - Solicitarle al juego un nuevo GameState y mostrar todo su contenido por pantalla.

INSTRUCCIONES

Para realizar las tareas que se solicitan en este examen debes disponer del proyecto Irrgarten, en Java y en Ruby, desarrollado hasta la práctica 3 inclusive.

Se recuerda que el examen se realiza obligatoriamente en los ordenadores del aula. Se debe arrancar con el código: `w10pdoovix` (en caso de trabajar con Windows y RubyMine) o `ubu20exhome` (en caso de preferir ejecutar Ruby desde terminal de Ubuntu).

Plazos:

- Se dispone de **15 minutos** para arrancar los ordenadores, leer el examen y preguntar todas las dudas relativas a la interpretación de las preguntas.
Durante esta fase de preguntas no se permite teclear nada.
- A continuación, se dispone de **1 hora** para contestar y entregar el examen.

Entrega:

- **Entregables:** Dos archivos, `java.zip` y `ruby.zip` con los proyectos COMPLETOS junto con las modificaciones realizadas en este examen.
- **Forma de entrega:** Subirlos a PRADO en la actividad que se indique en el examen y antes de que termine el plazo de tiempo indicado para ello.
 - Pueden subirse varias versiones del examen, en ese caso deben numerarse como `java1.zip`, `java2.zip`, `ruby1.zip`, `ruby2.zip`, etc. De modo que la versión más reciente está en el archivo con el número más alto. **Solo se corregirán las versiones más recientes de ambos proyectos.**
- **Se debe devolver la hoja de las preguntas al profesor.**

Penalizaciones:

- Se penalizará con hasta un 5% de la puntuación total del examen por errores de diseño del proyecto.
- Además de las penalizaciones propias por no contestar correctamente las preguntas. Se penalizará cada ejercicio por errores que impidan compilar el código o hagan abortar el programa, según la siguiente secuencia:
 - Un 5% por el primer error.
 - Un 10% por cada error a partir del segundo.